



Details of Lock Requests and Deadlocks

Version 2026.1
2026-04-20

Details of Lock Requests and Deadlocks

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

- Details of Lock Requests and Deadlocks..... 1**
 - 1 Waiting Lock Requests 1
 - 2 Queuing of Array Node Lock Requests 2
 - 3 ECP Local and Remote Lock Requests 2
 - 4 Avoiding Deadlock 3
 - 5 See Also 4

Details of Lock Requests and Deadlocks

This page provides detailed information on how [lock](#) requests are handled in InterSystems IRIS® data platform, as well as a detailed look at deadlock scenarios.

1 Waiting Lock Requests

When a process holds an exclusive lock, it causes a wait condition for any other process that attempts to acquire the same lock, or a lock on a higher level node or lower level node of the held lock. When locking subscripted globals (array nodes) it is important to make the distinction between what you lock, and what other processes can lock:

- *What you lock:* you only have an explicit lock on the node you specify, not its higher or lower level nodes. For example, if you lock `^student(1,2)` you only have an explicit lock on `^student(1,2)`. You cannot release this node by releasing a higher level node (such as `^student(1)`) because you don't have an explicit lock on that node. You can, of course, explicitly lock higher or lower nodes in any sequence.
- *What they can lock:* the node that you lock bars other processes from locking that exact node or a higher or lower level node (a parent or child of that node). They cannot lock the parent `^student(1)` because to do so would also implicitly lock the child `^student(1,2)`, which your process has already explicitly locked. They cannot lock the child `^student(1,2,3)` because your process has locked the parent `^student(1,2)`. These other processes wait on the lock queue in the order specified. They are listed in the lock table as waiting on the highest level node specified ahead of them in the queue. This may be a locked node, or a node waiting to be locked.

For example:

1. Process A locks `^student(1,2)`.
2. Process B attempts to lock `^student(1)`, but is barred. This is because if Process B locked `^student(1)`, it would also (implicitly) lock `^student(1,2)`. But Process A holds a lock on `^student(1,2)`. The lock Table lists it as `WaitExclusiveParent ^student(1,2)`.
3. Process C attempts to lock `^student(1,2,3)`, but is barred. The lock Table lists it as `WaitExclusiveParent ^student(1,2)`. Process A holds a lock on `^student(1,2)` and thus an implicit lock on `^student(1,2,3)`. However, because Process C is lower in the queue than Process B, Process C must wait for Process B to lock and then release `^student(1)`.
4. Process A locks `^student(1,2,3)`. The waiting locks remain unchanged.
5. Process A locks `^student(1)`. The waiting locks change:
 - Process B is listed as `WaitExclusiveExact ^student(1)`. Process B is waiting to lock the exact lock (`^student(1)`) that Process A holds.
 - Process C is listed as `WaitExclusiveChild ^student(1)`. Process C is lower in the queue than Process B, so it is waiting for Process B to lock and release its requested lock. Then Process C will be able to lock the child of the Process B lock. Process B, in turn, is waiting for Process A to release `^student(1)`.
6. Process A unlocks `^student(1)`. The waiting locks change back to `WaitExclusiveParent ^student(1,2)`. (Same conditions as steps 2 and 3.)
7. Process A unlocks `^student(1,2)`. The waiting locks change to `WaitExclusiveParent ^student(1,2,3)`. Process B is waiting to lock `^student(1)`, the parent of the current Process A lock `^student(1,2,3)`. Process C is

waiting for Process B to lock then unlock $\wedge\text{student}(1)$, the parent of the $\wedge\text{student}(1, 2, 3)$ lock requested by Process C.

8. Process A unlocks $\wedge\text{student}(1, 2, 3)$. Process B locks $\wedge\text{student}(1)$. Process C is now barred by Process B. Process C is listed as `WaitExclusiveChild  $\wedge\text{student}(1)$` . Process C is waiting to lock $\wedge\text{student}(1, 2, 3)$, the child of the current Process B lock.

2 Queuing of Array Node Lock Requests

The basic queuing algorithm for array locks is to queue lock requests for the same resource strictly in the order received, even when there is no direct resource contention. This is illustrated in the following example, in which three locks on the same global array are requested by three different processes in the sequence shown:

```
Process A: LOCK  $\wedge x(1, 1)$ 
Process B: LOCK  $\wedge x(1)$ 
Process C: LOCK  $\wedge x(1, 2)$ 
```

The status of these requests is as follows:

- Process A holds a lock on $\wedge x(1, 1)$.
- Process B cannot lock $\wedge x(1)$ until Process A releases its lock on $\wedge x(1, 1)$.
- Process C is also blocked, but not by Process A's lock; rather, it is the fact that Process B is waiting to explicitly lock $\wedge x(1)$, and thus implicitly lock $\wedge x(1, 2)$, that blocks Process C.

This approach is designed to speed the next job in the sequence after the one holding the lock. Allowing Process C to jump Process B in the queue would speed Process C, but could unacceptably delay Process B, especially if there are many jobs like Process C.

The exception to the general rule that requests are processed in the order received is that a process holding a lock on a parent node is immediately granted any requested lock on a child of that node. For example, consider the following extension of the previous example:

```
Process A: LOCK  $\wedge x(1, 1)$ 
Process B: LOCK  $\wedge x(1)$ 
Process C: LOCK  $\wedge x(1, 2)$ 
Process A: LOCK  $\wedge x(1, 2)$ 
```

In this case, Process A is immediately granted the requested lock on $\wedge x(1, 2)$, ahead of both Process B and Process C, because it already holds a lock on $\wedge x(1, 1)$.

Note: This process queuing algorithm applies to all subscripted lock requests. However, the release of a nonsubscripted lock, such as `LOCK  $\wedge x$` , when there are both nonsubscripted (`LOCK  $\wedge x$`) and subscripted (`LOCK  $\wedge x(1, 1)$`) requests waiting is a special case, in which the lock request granted is unpredictable and may not follow process queuing.

3 ECP Local and Remote Lock Requests

When releasing a lock, an ECP client may donate the lock to a local waiter in preference to waiters on other systems in order to improve performance. The number of times this is allowed to happen is limited in order to prevent unacceptable delays for remote lock waiters.

4 Avoiding Deadlock

Requesting a (+) exclusive lock when you hold an existing shared lock is potentially dangerous because it can lead to a situation known as "deadlock". This situation occurs when two processes each request an exclusive lock on a lock name already locked as a shared lock by the other process. As a result, each process hangs while waiting for the other process to release the existing shared lock.

The following example shows how this can occur (numbers indicate the sequence of operations):

Process A	Process B
1. LOCK ^a(1)#"S" Process A acquires shared lock.	
	2. LOCK ^a(1)#"S" Process B acquires shared lock.
3. LOCK +^a(1) Process A requests exclusive lock and waits for Process B to release its shared lock.	
	4. LOCK +^a(1) Process B requests exclusive lock and waits for Process A to release its shared lock. Deadlock occurs.

This is the simplest form of deadlock. Deadlock can also occur when a process is requesting a lock on the parent node or child node of a held lock.

To prevent deadlocks, request the exclusive lock without the plus sign, which unlocks your shared lock. In the following example, both processes release their prior locks when requesting an exclusive lock to avoid deadlock (numbers indicate the sequence of operations). Note which process acquires the exclusive lock:

Process A	Process B
1. LOCK ^a(1)#"S" Process A acquires shared lock.	
	2. LOCK ^a(1)#"S" Process B acquires shared lock.
3. LOCK ^a(1) Process A releases shared lock, requests exclusive lock, and waits for Process B to release its shared lock.	
	4. LOCK ^a(1) Process B releases shared lock and requests exclusive lock. Process A immediately acquires its requested shared lock. Process B waits for Process A to release its shared lock.

Another way to avoid deadlocks is to follow a strict protocol for the order in which you issue **LOCK +** and **LOCK -** commands. Deadlocks cannot occur as long as all processes follow the same order. A simple protocol is for all processes to apply and release locks in collating sequence order.

To minimize the impact of a deadlock situation, include the *timeout* argument when using plus sign locks. For example, the **LOCK +^a(1);10** operation times out after 10 seconds.

If a deadlock occurs, you can resolve it by using the Management Portal or the [^LOCKTAB utility](#) to remove one of the locks in question. From the Management Portal, open the **Manage Locks** window, and then select the **Remove** option for the deadlocked process.

5 See Also

- [Locking and Concurrency Control](#)
- [Managing the Lock Table](#)